

Task1

Q1

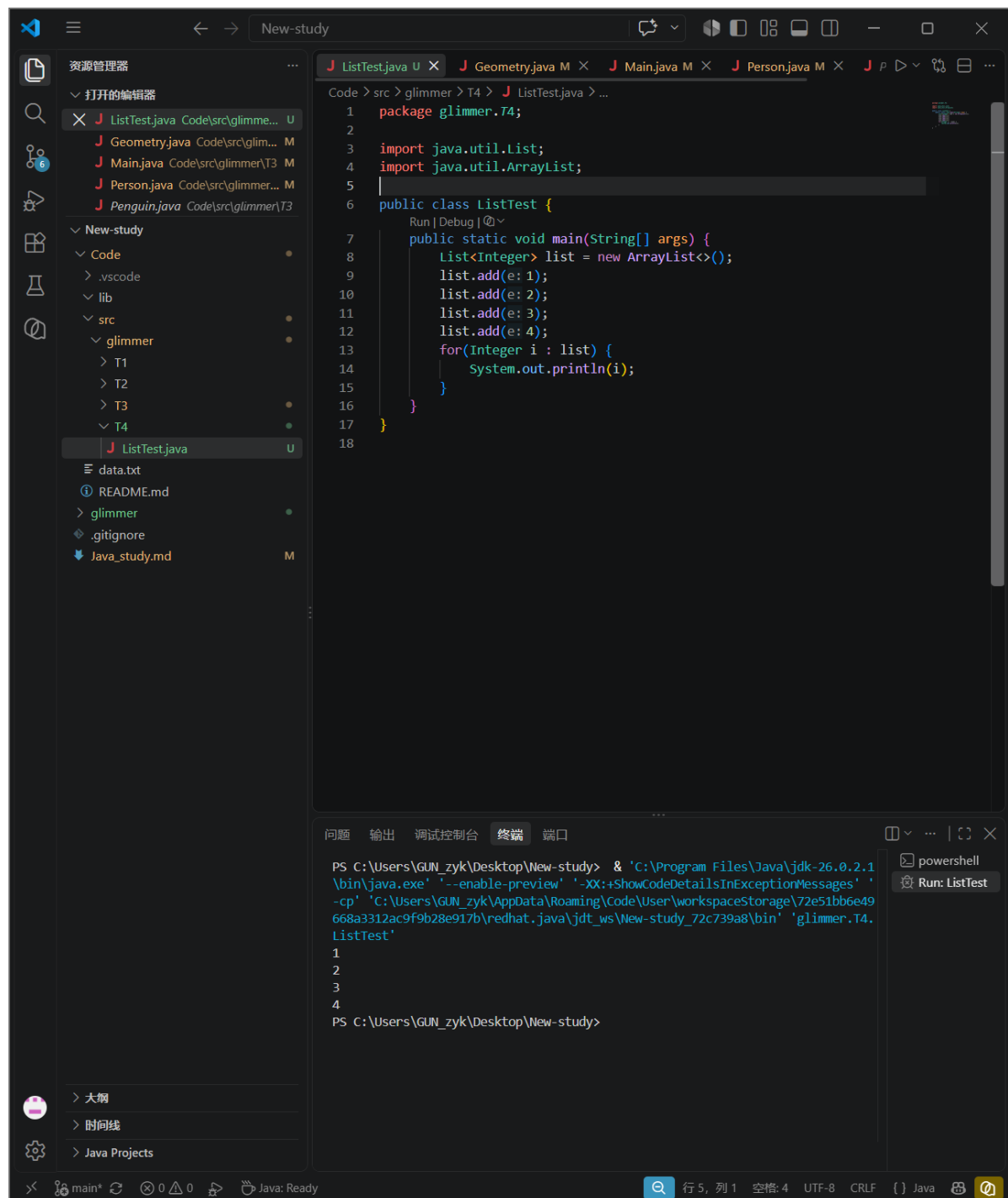
集合就是个vector版的数组/链表/set/map

集合下有list, set, map等多个接口, 实现类里包含TreeSet等(红黑树实现(想你了STL))

长度可变

但只能维护对象

Q2



```
Code > src > glimmer > T4 > J ListTest.java > ...
1  package glimmer.T4;
2
3  import java.util.List;
4  import java.util.ArrayList;
5
6  public class ListTest {
7      public static void main(String[] args) {
8          List<Integer> list = new ArrayList<>();
9          list.add(e: 1);
10         list.add(e: 2);
11         list.add(e: 3);
12         list.add(e: 4);
13         for(Integer i : list) {
14             System.out.println(i);
15         }
16     }
17 }
18
```

```
PS C:\Users\GUN_zyk\Desktop\New-study> & 'C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\GUN_zyk\AppData\Roaming\Code\User\workspaceStorage\72e51bb6e49668a3312ac9f9b28e917b\redhat.java\jdt_ws\New-study_72c739a8\bin' 'glimmer.T4.ListTest'
1
2
3
4
PS C:\Users\GUN_zyk\Desktop\New-study>
```

Q3

内部类

定义在另一个类地内部

内部类不能有静态方法和变量

静态内部类

也叫嵌套类

加了 `static` 的内部类，属于外部类本身，不属于外部类实例

1. 静态内部类**不能直接访问外部类非 static 成员**，只能访问静态成员
2. 创建对象不需要外部类实例：`外部类.静态内部类 对象 = new 外部类.静态内部类();`

局部内部类

定义在方法里面，作用范围仅限当前这个方法。

Java8 起，可以访问方法里的局部变量

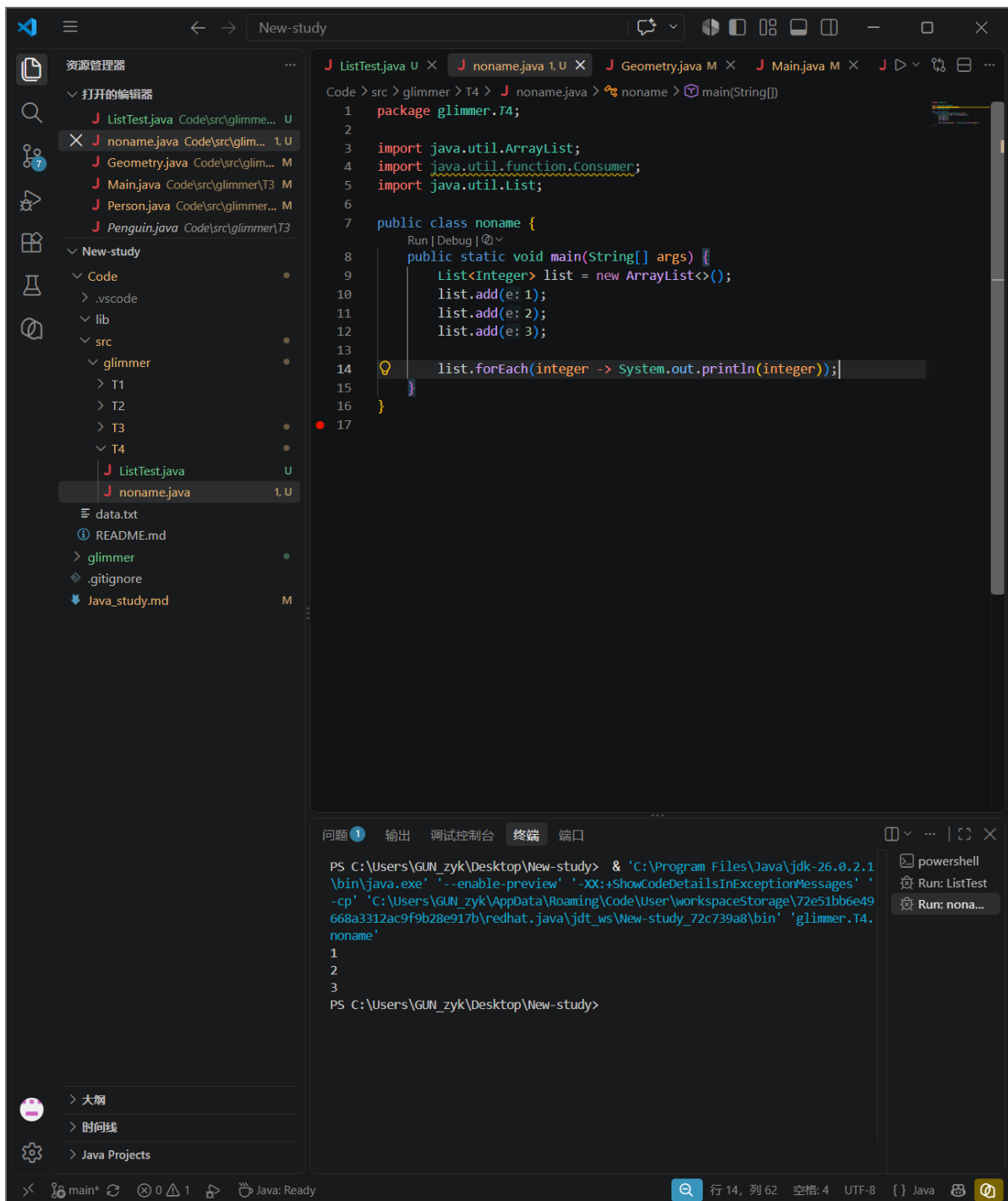
匿名内部类

现场重写，只用一次（只用在对象）

eg:

```
// 一个接口
interface Animal {
    void cry();
}

public class Test {
    public static void main(String[] args) {
        // 匿名内部类: new 接口, 同时写实现代码
        Animal cat = new Animal() {
            @Override
            public void cry() {
                System.out.println("喵喵");
            }
        };
        cat.cry();
    }
}
```



Task2

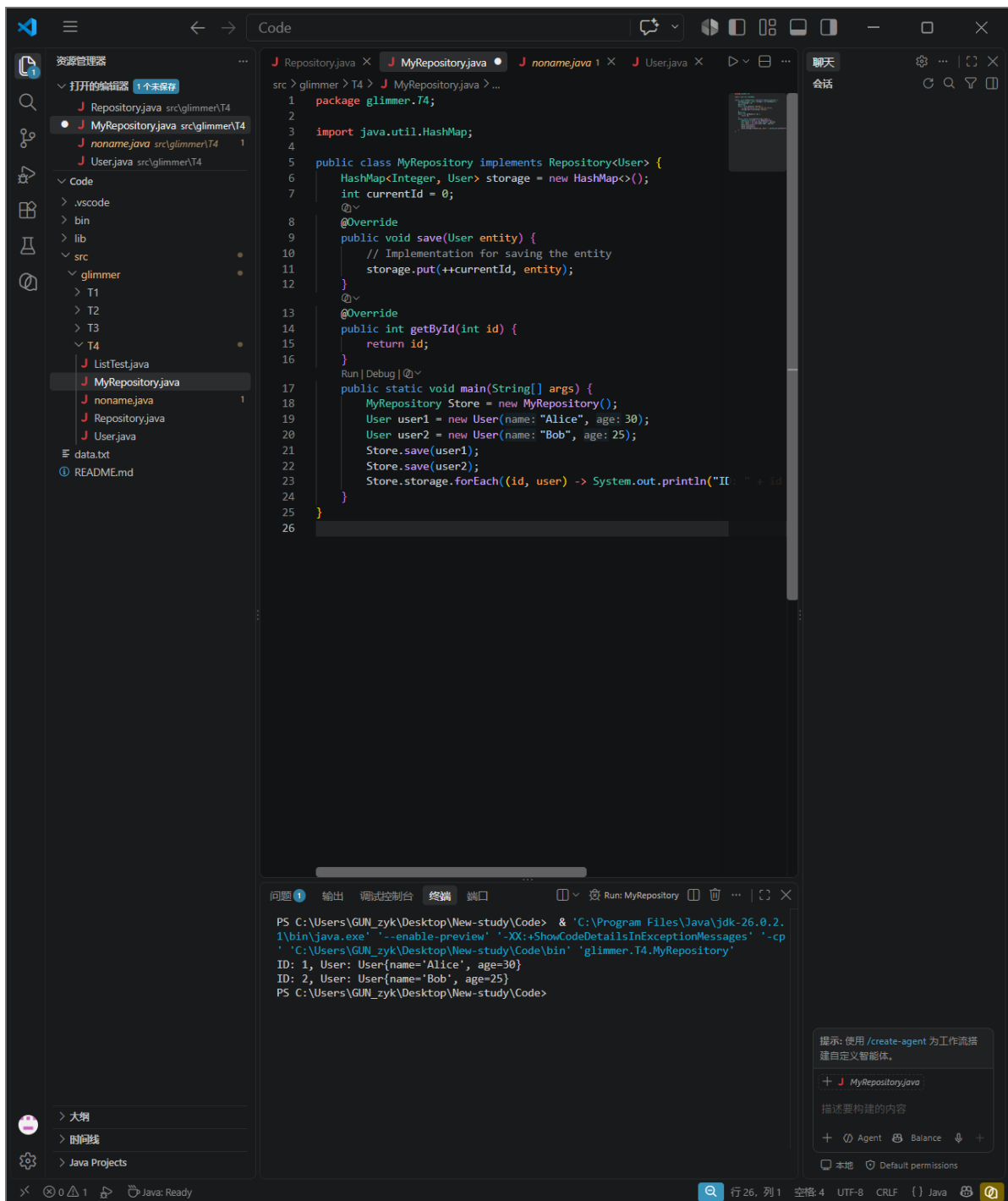
泛型像是高级object

Q4

接口里不能有实例成员

数据类型为user，对象在MtRepository里

Map.foreach是BiConsumer接口，有两个参数



Q5

简单模式：常用于配置管理、日志记录、数据库连接池等场景，**全局只有一个对象**

工厂模式：客户端不需要知道怎么new对象，只要告诉工厂要干什么，让工厂分辨new哪一个对象（有多个对象有相同性质）

建造者模式：new对象的过程中有很多初始化参数，由build统一步骤，调用多个方法调整参数

学长玩fgo?

```

package glimmer.T4;

public class Caster implements Character {
    private String name;
    private String skill;

    public Caster() {
        this.name = "美狄亚";
        this.skill = "Rho Aias";
    }

    public void attack() {
        System.out.println("[法师] " + name + " 使用 " + skill + " 发动攻击!");
    }
}

```

```

package glimmer.T4;

public class Archer implements Character{
    private String name;
    private String skill;

    public Archer() {
        this.name = "卫宫";
        this.skill = "Unlimited Blade Works";
    }

    public void attack() {
        System.out.println("[弓兵] " + name + " 使用 " + skill + " 发动攻击!");
    }
}

```

```

package glimmer.T4;

public class Saber implements Character{
    private String name;
    private String skill;
    public Saber() {
        this.name = "阿尔托莉雅";
        this.skill = "Excalibur";//咖喱棒
    }

    public void attack() {
        System.out.println("[剑士] " + name + " 使用 " + skill + " 发动攻击!");
    }
}

```

```

package glimmer.T4;

public interface Character {
    void attack();
}

```

```

package glimmer.T4;
enum CharacterType {
    SABER, ARCHER, CASTER
} // 学长玩fgo?
public class CharacterFactory {
    @
    public static Character createCharacter(CharacterType type) {
        switch (type) {
            case SABER:
                return new Saber();
            case ARCHER:
                return new Archer();
            case CASTER:
                return new Caster();
            default:
                throw new IllegalArgumentException("Invalid character type: " + type);
                // 这行是AI补全的，意思是入参不合法
        }
    }
}

Run | Debug |
public static void main(String[] args) {
    Character saber = CharacterFactory.createCharacter(CharacterType.SABER);
    saber.attack();

    Character archer = CharacterFactory.createCharacter(CharacterType.ARCHER);
    archer.attack();

    Character caster = CharacterFactory.createCharacter(CharacterType.CASTER);
    caster.attack();
}
}

```